1on1 --- Product Requirements & Product Vision

1. Product Definition

1on1 is an open professional networking and session platform.

It is not primarily a learning platform, coding platform, mentor marketplace, or course platform.

The product combines:

- Professional identity similar to LinkedIn
- Public professional content and engagement
- Open following instead of mutual connections
- Direct 1:1 and group session requests
- Free or paid sessions, with payments mocked for the current project
- Integrated WebRTC meeting rooms
- AI-powered meeting intelligence
- AI-powered platform assistant
- Skill/achievement/certification credibility
- Optional mentor/provider identity

The central product primitive is the **session**.

A user can discover another person, follow them, engage with their content, and request a session with them. The session can be free or paid depending on the provider's settings.

Becoming a mentor/provider is optional. A user does not need to become a mentor to use the platform.

---

2. Product Positioning

The simplest conceptual comparison is:

This is a conceptual comparison, not a requirement to copy those products.

The product identity should remain:

> **An open professional network where people can discover professionals, follow their work, engage through content, and directly request free or paid 1:1 or group sessions---with an integrated AI-powered meeting experience.**

---

3. Core Product Loop

The core loop is not "learn → prove → become mentor."

The primary loop is:

DISCOVER → FOLLOW → ENGAGE → REQUEST SESSION → MEET → COLLABORATE → RETURN / REVIEW / FOLLOW

A user can participate in this loop without ever becoming a mentor.

A separate optional path is:

Build profile → demonstrate expertise → offer sessions → become a mentor/provider

---

4. User Identity

Every user has a professional identity.

Profile sections can include:

- Name
- Profile picture
- Cover image
- About
- Skills
- Experience
- Education
- Certifications
- Projects
- Achievements
- Portfolio links
- Social links
- Posts
- Followers
- Following
- Session offerings
- Reviews
- Optional mentor/provider status
- Optional verification status

The profile is designed to answer:

1. Who is this person?
2. What do they know?
3. What have they worked on?
4. What have they achieved?
5. What do they post about?
6. Can I request a session with them?

---

5. Following Model

There are **no connection requests** and no mutual connection model.

The relationship is one-way:

User A → follows → User B

Supported actions:

- Follow
- Unfollow
- Remove follower
- Block
- Report

A user can follow someone without that person following back.

This makes the social graph simpler and closer to an open creator/professional network.

---

6. Public-First Product Philosophy

The platform should be public by default in the same general spirit as LinkedIn.

Public content can include:

- Profiles
- Posts
- Skills
- Experience
- Certifications
- Projects
- Public session offerings
- Public engagement

Privacy controls can be added later where required, but the first product should not become a complicated privacy-management system.

---

7. Posts and Feed

Users can publish professional content.

Post types:

- Text
- Images
- Videos
- Links
- Polls
- Code snippets

No file-sharing feature is required for the current product.

Users can:

- Like
- Comment
- Repost
- Bookmark
- Share
- Follow the author
- Report content

The feed is a professional social feed, not a course feed.

---

8. Feed Ranking --- V1

Do not build a large recommendation ML system initially.

V1 ranking can combine:

- Recency
- Engagement
- Follow relationship
- Skill relevance
- Basic user interests

Conceptually:

```
Feed Score = Recency + Engagement + Follow Relevance + Skill Relevance
```

Later, the system can evolve toward personalized ranking and recommendation models.

Open-source recommendation work from large platforms such as X can be used as architectural inspiration, but it should not be copied blindly. The initial user volume does not justify a large recommendation infrastructure.

---

9. Professional Search

Search is a major product feature.

Users should be able to discover people using:

- Name
- Skill
- Experience
- Company
- Education
- Location
- Certification
- Mentor/provider status
- Session availability
- Session price
- Rating

Example:

> Find React developers with 3+ years of experience who offer free sessions.

Later, natural-language search can be handled by the AI assistant.

---

10. Sessions

Sessions are the central product object.

A session can be:

- 1:1

- Group
- Free
- Paid
- Scheduled
- Completed
- Cancelled
- Rescheduled

There is no separate "session marketplace" product concept.

Instead, users publish their availability/session offerings and other users request access.

---

11. Session Request Flow

The normal flow is:

Session discovered → Request session → Select preferred time → Provider accepts/rejects → Scheduled → Reminder → Meeting room → Completed → AI notes → Review

Session states:

- REQUESTED
- ACCEPTED
- REJECTED
- RESCHEDULED
- CANCELLED
- IN_PROGRESS
- COMPLETED
- NO_SHOW

The server must be the source of truth for state transitions.

---

12. Preferred Time

A requester should be able to provide preferred times.

The provider can publish availability such as:

- Monday: 6 PM--8 PM
- Wednesday: 7 PM--10 PM
- Saturday: 10 AM--1 PM

The requester selects a preferred slot from available times.

This is a core feature because session scheduling is central to the product.

---

13. Session Types

Do not introduce a separate session marketplace taxonomy.

Instead, session providers can describe their sessions naturally.

Examples:

- 30-minute Career Discussion
- 60-minute React Architecture Discussion
- 45-minute Resume Review
- 30-minute Startup Advice
- 90-minute Group Discussion

Each session configuration can include:

- Title
- Description
- Duration
- Price
- Maximum participants
- Availability
- Requirements/instructions

---

14. Mock Payments

There will be **no real payment gateway** in the current application.

Paid sessions are simulated.

Example:

`Session price: ₹999`

→ Mock payment

→ Payment marked `PAID`

→ Session confirmed

The data model should still support:

- Amount
- Currency
- Payment status
- Refund amount
- Cancellation fee
- Cancellation policy
- Payment timestamp

Real Razorpay/Stripe integration is a future task, not part of the current implementation.

---

15. Cancellation Policy

Cancellation should be policy-driven.

Example prototype policy:

- More than 3 hours before session: 10% cancellation fee, 90% refund
- 1--3 hours before session: 20% cancellation fee, 80% refund
- Less than 1 hour: higher cancellation penalty according to configured policy
- Provider cancellation: 100% refund

The policy should be represented as data rather than hard-coded into UI.

Store:

- Cancellation timestamp
- Cancelled by
- Original booking amount
- Cancellation fee
- Refund amount
- Policy rule applied

The prototype only simulates the refund.

---

16. Meeting Room

The meeting room is a major differentiator.

Technology:

WebRTC

Meeting controls/features:

- Camera on/off
- Microphone on/off
- Speaker/audio controls
- Fullscreen
- Leave/end meeting
- Participant list
- Screen sharing
- Session chat
- Reactions
- Raise hand
- Whiteboard
- Shared notes
- Code snippets
- Polls
- Q&A
- Recording
- AI transcript
- AI summary
- Action items
- Meeting recap

No file sharing is required.

---

17. Whiteboard

The whiteboard is part of the meeting experience.

It is intended for:

- Explaining architecture
- Drawing diagrams
- Teaching concepts
- Brainstorming
- Interview discussions
- Product discussions
- Technical explanations

Participants can:

- Draw

- Write
- Add shapes
- Add arrows
- Erase
- Move objects

For V1, synchronization can use Socket.IO.

A future high-concurrency implementation could use CRDT-based collaboration.

---

18. Screen Sharing

Participants can share:

- Entire screen
- Window
- Browser tab

Screen sharing uses WebRTC.

This is useful for:

- Code walkthroughs
- Portfolio reviews
- Presentations
- Debugging
- Product demos
- Architecture discussions

---

19. Session Recording

Recording is supported in the product plan.

The implementation should eventually support:

- Start recording
- Stop recording
- Recording status
- Post-session recording availability
- Access control

Recording must require appropriate participant consent.

Storage and processing architecture can be added when cloud deployment becomes the focus.

---

20. AI Meeting Assistant

After/during a session, AI can process meeting information.

Pipeline:

Meeting audio → transcript → Gemini → structured meeting intelligence

Possible outputs:

- Summary
- Key points
- Decisions
- Questions discussed
- Action items
- People responsible
- Topics discussed
- Follow-up recommendations

Example:

> Action item: Tej will send the project architecture by Friday.

The AI recap becomes part of the session history.

---

21. AI Platform Assistant

A persistent AI button can appear at the top-left of the application.

Clicking it opens a left-side assistant panel.

The assistant should not merely answer questions. It should eventually perform platform tasks.

Examples:

> Find React developers with 3+ years experience.
>
> Find people I follow who posted about AI.
>
> Find available Python mentors tomorrow evening.

> Book me a session with Rahul at 7 PM.
>
> Cancel my session with Rahul.
>
> Show my upcoming sessions.
>
> Explain the cancellation policy.

This makes the assistant an **agentic interface to the platform**.

---

22. AI Agent V1 Scope

Keep V1 intentionally limited.

Initial tools:

- searchUsers
- searchPosts
- searchSessions
- getAvailability
- requestSession
- cancelSession
- rescheduleSession
- followUser
- unfollowUser
- sendMessage
- getNotifications

Destructive actions require confirmation.

For example:

User: "Cancel my session."

Agent:

> I found your session with Rahul at 7 PM. Cancelling now will result in an ₹X mock refund. Confirm?

Only after confirmation should the tool execute.

---

23. AI Agent Security

The Gemini model must never receive direct database access.

Correct architecture:

User → Agent → Gemini → Tool → Express authorization → MongoDB

Every tool must validate:

- Authenticated user
- Authorization
- Resource ownership
- Allowed operation
- Input schema
- Session state
- Business rules

The agent should never be trusted as the authorization layer.

---

24. AI Moderation

No admin moderation dashboard is required.

AI moderation handles spam and unsafe content.

Potential checks:

- Spam
- Scam
- Harassment
- Abusive content
- Malicious links
- Promotional spam
- Prompt injection attempts

V1 decision model:

- Low risk → allow
- Medium risk → warning/additional check
- High risk → block

Serious moderation actions should be conservative and auditable even without an admin UI.

---

25. Certifications and SHA-256

Certification verification is part of the current product plan.

Do not call SHA-256 itself blockchain.

The product can generate a cryptographic certificate fingerprint:

Certificate data → canonical representation → SHA-256 hash → verification ID → public verification page

Blockchain anchoring is part of the current plan as a later implementation layer for certification integrity.

The blockchain should store/anchor the certificate hash rather than storing sensitive certificate data directly.

---

26. Optional Mentor/Provider Status

Becoming a mentor/provider is optional.

A user can simply:

- Build a profile
- Follow people
- Post
- Engage
- Request sessions
- Participate in group sessions

Or they can additionally offer sessions.

Technical achievements, coding practice, badges and certifications can help establish credibility, but they do not force the user into a mentor path.

---

27. Coding and Achievement Layer

The coding/achievement system is supportive rather than the product's central identity.

It can include:

- Coding problems
- Timed assignments
- Solved count
- Attempts
- Badges

- Skill evidence
- Achievement history

These can improve a professional profile and help establish credibility for users who choose to offer sessions.

---

28. Notifications

Notifications should cover:

- New follower
- New comment
- New like
- Repost
- Session request
- Session accepted
- Session rejected
- Session rescheduled
- Session reminder
- Session cancelled
- Session completed
- New message
- Mention
- AI task completion

Realtime notification delivery uses Socket.IO.

Persistent notifications are stored in MongoDB.

---

29. No Admin Dashboard in Current Scope

There is intentionally no admin dashboard in the current product.

No:

- Admin analytics
- User management dashboard
- Problem management dashboard
- Admin moderation UI

The product is intended to be fully dynamic for normal users.

---

30. Product Principles

1. Professional identity first.
2. Open discovery.
3. Follow, not connection requests.
4. Sessions are the central interaction primitive.
5. Free and paid sessions can coexist.
6. Group sessions exist from the beginning.
7. Meeting experience is a first-class product.
8. AI is integrated into the product rather than bolted on as a chatbot.
9. Mentor/provider status is optional.
10. Keep V1 technically simple and expandable.